

Kalakriti

How five people build this without standing on each other

A module split with named owners, exact file boundaries, the contracts between modules, and a rule for the four files everybody has to touch.

Read this first

The split is by **vertical slice**, not by frontend and backend. Each module owns its screens *and* the endpoints behind them, so one person can finish a feature without waiting for somebody else to expose an API. That is the whole reason the boundaries fall where they do.

What already works, so nobody rebuilds it

Working today	Where
Real OTP login, sessions, logout that actually revokes	backend/auth.py, routes_auth.py
Persistence that survives closing the app; offline queue with field-level conflict resolution	app/src/lib/{storage,session,sync,store}
Nine languages, 512 strings, no network needed to render them	app/src/i18n/
A real storefront: public product page, working Buy button, real order rows	backend/main.py, channels.py
Counted storefront views; every other marketplace metric reports <i>unavailable</i> with the platform's reason	backend/analytics.py
Auto mode (server-side jobs) and manual mode (per-field AI)	backend/jobs.py, app/src/ui/AiButton.tsx
Guided in-app camera with live framing and quality checks	app/src/screens/GuidedCamera.native.tsx, src/vision/

The five modules at a glance

#	Module	Owns	The one sentence
M1	Capture & on-device vision	camera, image quality, OCR, matting	Get a good photograph out of a cheap phone in bad light.
M2	Listing studio & AI	Create screen, jobs, per-field assist	Turn that photograph into a listing she is happy to put her name on.
M3	Identity & seller readiness	onboarding, auth, profile, KYC	Make her a seller a marketplace will actually accept.
M4	Commerce	products, orders, marketplaces, enquiries	Everything after Publish: where it went, who bought it, what happened.
M5	Platform	storage, sync, i18n, hosting, design	Make all of it survive a bad connection and run somewhere public.

1. Who owns which files

If a file is not on your list, do not edit it. Open an issue and tag the owner. The four shared files have their own rule in section 3.

M1 - Capture and on-device vision

Owns	Path
Guided camera, all platforms	app/src/screens/GuidedCamera*.tsx
Live framing, blur, exposure, tilt	app/src/vision/*.ts
Per-craft shot plan	app/src/screens/shotPlan.ts
Enhancement, matting	backend/imaging.py
Background generation	backend/bg.py
OCR and vision-model reading	backend/vision.py

M2 - Listing studio and AI assist

Owns	Path
The whole create flow	app/src/screens/Create.tsx
Auto / manual chooser	app/src/screens/ModeSheet.tsx
Per-field AI button and sheet	app/src/ui/AiButton.tsx
Connection measurement	app/src/lib/connection.ts
Server-side job queue	backend/jobs.py
The shared image pipeline	backend/pipeline.py
Copy and pricing prompts	backend/llm.py
Routes: /v1/jobs*, /v1/assist*, /v1/analyze, /v1/catalog, /v1/price	backend/main.py (own section)

M3 - Identity, profile and seller readiness

Owns	Path
Language choice and onboarding	app/src/screens/Onboarding.tsx, LanguageSheet.tsx
Login, OTP, progressive profile	app/src/screens/AuthSheet.tsx
Profile tab	app/src/screens/ProfileTab.tsx
Tokens, sessions, OTP challenges	backend/auth.py
Auth and profile routes	backend/routes_auth.py
SMS delivery	backend/sms.py
Per-marketplace field mapping and readiness	backend/seller.py

M4 - Commerce

Owns	Path
Home dashboard	app/src/screens/HomeTab.tsx
Product list, detail, marketplace detail	app/src/screens/{ProductsTab,ProductDetail,MarketplaceDetail}.tsx
Orders and bulk enquiries	app/src/screens/{OrdersTab,Enquiries}.tsx
Filter and sort bar, metric tiles	app/src/ui/{Filters,Metric}.tsx
Marketplace adapters and live stats	backend/channels.py
Read models: cards, summary, insights, views	backend/analytics.py
Couriers, tracking, webhooks	backend/logistics.py
Routes: /v1/listings*, /v1/orders*, /v1/enquiries*, /v1/summary, /v1/insights, /webhooks/*	backend/main.py (own section)

M5 - Platform

Owns	Path
Device storage, session, cache, offline queue, data layer	app/src/lib/{storage,session,cache,sync,store,config}.*
Navigation shell, tab bar, top bar	app/src/nav/Shell.tsx
Design system and shared components	app/src/theme.ts, app/src/ui.tsx
Translation pipeline and script checking	backend/translate_ui.py, check_scripts.py, app/scripts/
Schema and migrations	backend/db.py
Object storage	backend/media.py
Hosting, CI, native build	Dockerfile, render.yaml, app/plugins/, app/app.json

2. What each module has left to do

Ordered so the first item is worth starting on Monday morning. **P0** blocks a demo, **P1** is the visible win, **P2** is depth.

M1 - Capture and on-device vision

P	Task	Done when
P0	Multi-photo capture completes the shot plan and all photos reach the listing, not only the first	A pot photographed front, top and base shows three images on the product page.
P1	Craft-specific guidance for at least four crafts beyond textiles (pottery, metal, wood, jewellery)	The overlay and the spoken hint change with the craft, verified on device.
P1	Quality gate before upload: refuse a photograph too blurred or too dark to read, and say which	A deliberately blurred shot is rejected with a reason, not silently analysed.
P2	Cut the upload: downscale and re-encode on the phone before sending	Median upload under 150 KB with no visible loss on the storefront page.
P2	An on-device object check so the guidance works with no signal at all	Framing feedback still moves with aeroplane mode on.

M2 - Listing studio and AI assist

P	Task	Done when
P0	Progressive delivery: fetch fields, then thumbnail, then full image, each only when the connection allows	On a throttled connection the listing is editable before any image arrives.
P1	AI buttons on the remaining fields - category, HSN, tags, dimensions	Every field in the form has a working AI button and a Keep mine option.
P1	Job retry from the failure card, reusing the stored upload	A job failed by killing the server can be retried without re-uploading.
P2	Voice note in nine languages, not only Hindi	A Gujarati voice note produces a Gujarati description.
P2	Bulk mode: photograph several products in one session	Five photos become five drafts from one trip to the camera.

M3 - Identity, profile and seller readiness

P	Task	Done when
P0	Real SMS through MSG91, with a DLT template	A code arrives on a phone that is not the developer's, with OTP_DEV_ECHO off.
P1	Profile completeness meter driven by what each marketplace actually requires	The bar moves as fields are filled and names the channel each one unlocks.
P1	GSTIN and IFSC validated by format and checksum before submission	A malformed GSTIN is caught in the app, not by GeM three days later.
P2	Account recovery when the phone number changes	A documented, tested path that does not involve editing the database.
P2	Aadhaar-linked artisan ID verification, if a government API is available	Either a working integration or a written note on which API and why not.

M4 - Commerce

P	Task	Done when
P0	One external marketplace fully live end to end - Shopify is the realistic first, its API is open and needs no seller approval	A product published from the app appears on a real Shopify store, and an order placed there appears in the Orders tab.
P1	Order detail screen: one order, full timeline, buyer, courier, refund	Tapping an order opens it; every field shown is from a real row.
P1	Return and refund flow through Razorpay	A test refund moves the order to refunded and the money is really returned.
P1	Enquiry to order: turn an accepted bulk enquiry into a real order	Accepting a 120-piece enquiry creates an order with the agreed price.
P2	Live courier tracking on the order card via webhook	A Shiprocket status change updates the app without a refresh.
P2	Export orders as CSV for accounting	A file that opens in Excel with correct Devanagari.

M5 - Platform

P	Task	Done when
P0	Deploy: Postgres, object storage, public HTTPS. See docs/DEPLOY.md	/health reports postgres and s3, and the APK works on mobile data with the laptop switched off.
P0	Finish Telugu, Kannada and Odia - all three are still largely English	check_scripts.py is clean and every language is above 95% translated.
P1	Push notification when a server-side listing is ready	The phone buzzes with the app closed. Needs an FCM project.
P1	CI: typecheck, backend tests and an APK build on every push	A pull request that breaks the build cannot be merged.
P2	Alembic, before the first schema change on live data	A migration runs against a copy of production without losing a row.
P2	Native-speaker review of the generated translations	A first-language reader has signed off each language.

3. The four files everybody has to touch

These cannot be owned by one person without blocking everyone. They have a rule instead. Follow it and git will merge your work without a conflict; ignore it and you will spend Friday resolving one.

File	Rule
app/src/i18n/catalog.ts	Append only, inside your own comment block. Add to EN and HI in the same commit, keeping the keys in the same order. Never reformat or reorder anybody else's block. Prefix every key with your surface: cam., create., auth., prod., sync.
app/src/lib/api.ts	Append your endpoints and types at the end, under a comment naming your module. Do not change an existing signature - add a new function instead and tell the owner.
backend/main.py	Every module has a numbered section. Add routes to your section only, at its end. Shared helpers go in a module file, never inline here.
app/App.tsx	M5 owns it. Need a new screen in the navigation? Send M5 the component and the props it wants. Two people editing this file is the single most likely merge conflict on the project.

Why key prefixes matter more than they look

All 512 strings live in one catalogue and nine language files are generated from it. A key that collides silently overwrites somebody else's string in every language at once, and the person who notices will be a Gujarati speaker three weeks later. The prefix is what makes a collision impossible.

Before you commit

```
cd app && ./node_modules/.bin/tsc --noEmit      # must be silent
cd app && node scripts/dump-en.mjs              # only if you added strings
cd backend && python check_scripts.py            # only if you touched locales
cd backend && python -c "import main"           # must import cleanly
```

4. The contracts between modules

Where one module hands work to another. Agree a change here in the group before writing it, because both sides break at once.

From	To	Contract
M1	M2	pipeline.analyse_into() returns detected fields, confidences and suggestions. M2 fills blanks only and never overwrites what the artisan typed.
M2	M4	A listing reaches status='draft' with a price and a title before it can be published. M4 owns everything from Publish onwards.
M3	M4	seller.readiness(artisan, channel) decides whether publishing is allowed. M4 calls it and renders the result; it never re-implements the rules.
M5	all	useStore() is the only way a screen reads data. Do not call the API from a screen - that is how the offline queue gets bypassed and edits get lost.
M5	all	t('your.key') is the only way text reaches the screen. A literal string in a component is a bug, not a shortcut.

The rule that keeps the whole thing honest

No module invents a number. If a marketplace does not expose a metric, return `{value: null, available: false, why: "..."}` and the app renders a dash with the reason. A zero says nobody looked; the truth is nobody counted, and those are different claims. This is the thing judges will probe hardest, and it is the thing that separates this from every other cataloguing demo.

5. How to work in parallel without tripping

Branches

main	always builds, always demoable
m1/multi-photo	one branch per task, prefixed with your module
m2/progressive-delivery	
m4/shopify-live	

Rebase on main every morning. A branch that lives longer than three days is a merge conflict growing quietly.

Everyone can run everything

Nobody should be blocked waiting for somebody else's laptop. Each teammate runs the full stack:

```
cd backend && ./venv/Scripts/python.exe -m uvicorn main:app --host 0.0.0.0 --port 8000
cd app && npx expo start --web
```

The only shared thing is the NVIDIA key, and it has no per-seat limit. Once M5 has the hosted backend up, point at it with EXPO_PUBLIC_API_URL instead - then only the person changing the backend needs to run one.

Suggested first week

Day	M1	M2	M3	M4	M5
1	Read src/vision/, run the camera on a real phone	Read Create.tsx end to end	Open the MSG91 account - it needs DLT approval, start it today	Open a Shopify partner store	Deploy Postgres and R2
2-3	Multi-photo capture	Progressive delivery	Real SMS working	Shopify publish	Public HTTPS + finish the three languages
4-5	Craft guidance	Remaining AI buttons	Completeness meter	Shopify orders back into the app	CI, then push notifications
Demo	One good photo, badly lit, on a cheap phone	Manual mode, then Carry on with AI	Sign in on a phone nobody has seen	A real listing on a real Shopify store	Aeroplane mode, edit, reconnect, it syncs

Start the paperwork on day one

ONDC registration takes weeks, MSG91 needs DLT template approval, Razorpay needs KYC, Amazon needs a Professional selling account. None of it is code and all of it blocks a demo. The person who opens those accounts on Monday is worth more to this project in that hour than in the rest of the week.

What each person should be able to show at the end

- **M1** - photograph a real object in bad light on a cheap phone and get a usable listing image, with the guidance visibly helping.
- **M2** - the same product built twice: once by the AI from one photo, once by hand with AI help on two fields. Both editable, neither pretending.
- **M3** - a stranger signs in on their own phone with a real SMS, and the app says exactly which detail each marketplace still needs.

- **M4** - a product live on a real external marketplace, an order arriving from it, and a metrics screen that says plainly which numbers that platform refuses to give.
- **M5** - aeroplane mode, edit a price, come back, watch it sync; and the whole thing running on a public URL with the laptop shut.